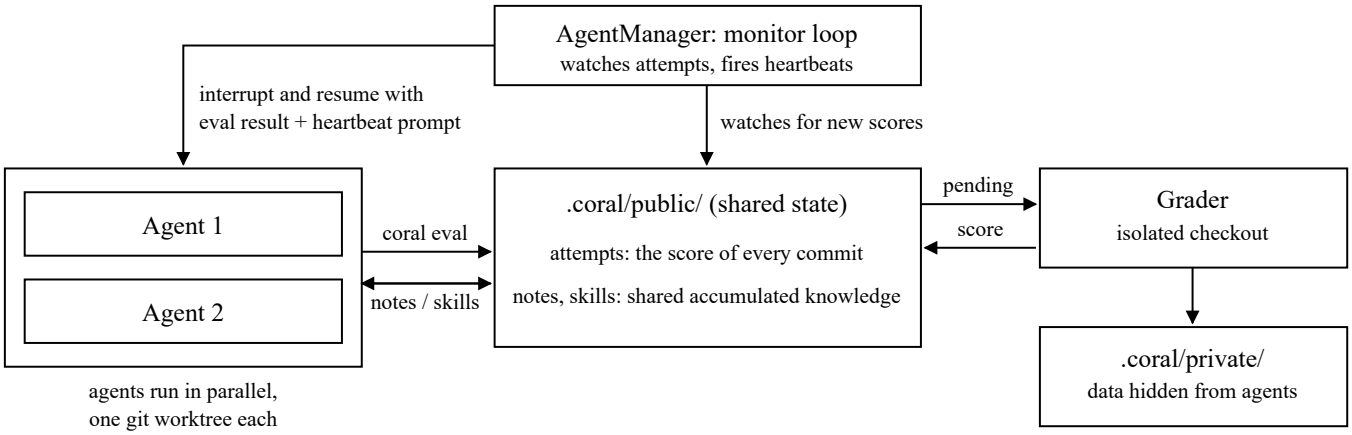
Alpha Z Technical Assessment: CORAL

The experiments and reports were done with the help of Claude Code.

What I understand about CORAL

CORAL is a harness that evolves a program. It launches N coding agents as ordinary CLI subprocesses, each working on its own git branch of the same seed program. An agent decides for itself when it is ready to be judged; that commits its code and files a pending attempt. A grader picks the attempt up, checks the commit out into an isolated worktree, runs the task's grader against data the agent cannot see, and writes the score back. A manager process watches for newly scored attempts and, on a cadence measured in evaluations rather than seconds, interrupts an agent mid-session and resumes it with the eval result plus an instruction to reflect, consolidate or pivot. Two things therefore accumulate: the program, as commits ranked by score, and a shared directory of markdown notes and skills that the agents write and read at will.



Task 1: Replicate a task result

Task `circle_packing` (26 circles in a unit square, maximise the sum of radii), Claude Opus 4.8, unmodified upstream grader and seed.

Final Score	**1.000002**. Sum of radii 2.635983084918, from a seed measured at 0.364102 (sum of radii 0.959764).
#Evals	**2**. The run auto-stopped on `score_threshold` after roughly 54 minutes.

Method	Sum of radii	Score	#Evals
Seed program (measured)	0.959764	0.364102	n/a
ShinkaEvolve	2.6001	0.986389	62
OpenEvolve	2.6293	0.997467	100
EvoX	2.6320	0.998491	48
CORAL (paper)	2.6360	1.000009	11
CORAL (this run)	**2.635983**	**1.000002**	**2**
AlphaEvolve (SOTA)	2.635977	1.000000	n/a

Score = sum of radii / 2.635977. Baselines are the paper's Table 1 (single-agent, Claude Opus 4.6).

Reading of the numbers. The run beats the benchmark by 6.085e-6 in sum of radii, which is the same order as the rounding in the grader's hardcoded constant `BENCHMARK = 2.635977`. The claim is therefore that CORAL matched AlphaEvolve to six significant figures, not that it beat it. Because the winning margin is smaller than the grader's own constraint tolerance of 1e-6, feasibility was re-checked at machine precision: the worst constraint violation is 6.106e-15.

Task 2: Apply CORAL to a classic OR problem

Problem chosen: Capacitated Vehicle Routing Problem (CVRP). A fleet of identical vehicles, each of capacity *Q*, leaves a single depot, serves every customer exactly once, and returns; the load on any route may not exceed *Q*. The objective is to minimise total distance travelled. Every instance has 100 customers, distances are EUC_2D (Euclidean, rounded to the nearest integer), and the fleet is effectively unlimited, so the solver chooses how many routes to open. The solver gets 10 seconds per instance.

Instances were generated with the XML100 generator published alongside CVRPLIB (galgos.inf.puc-rio.br/cvrplib/uploads/files/xml100/generator.py), described in Uchoa et al. (2017), *New benchmark instances for the Capacitated Vehicle Routing Problem*, EJOR, and Queiroga et al. (2022), *10,000 optimal CVRP solutions for testing machine learning based heuristics*. 60 instances were generated, stratified across the generator's 4 factors (depot position, customer position, demand distribution and average route size), then split into 50 hidden instances for grading and 10 disjoint public instances for development. They were generated fresh rather than taken from CVRPLIB Set X because Set X's optimal routes are published and its `dimension` field uniquely identifies each instance, so an agent with web access could hardcode a lookup table and score near 1.0 without solving anything.

`grader.py` loads the hidden instances, calls the agent's `solve()` once per instance, rejects any solution that revisits a customer, misses one, overloads a route or opens too many, and scores the run as the mean of `reference_distance / solver_distance` over the 50 instances. The reference distance for each instance is PyVRP's hybrid genetic search, best of 3 seeds at 60 seconds each, with every label independently re-validated against the grader's own feasibility rules. The seed program is a greedy nearest-neighbour construction with no improvement step. The run uses 2 agents on `deepseek-v4-flash` (set at launch and verified from the agent logs) with a fixed budget of 12 evaluations and no score threshold; `task.yaml` states the problem, the interface and the rules without naming any algorithm, library or reference solver.

Final Score **0.985405**. Mean gap +1.50% over the reference, up from the seed's 0.789815.

#Evals **12**. The run stopped on `max_real_attempts` after 1 h 46 min.

Solver	Score	Mean gap	Budget per instance
Seed: greedy nearest neighbour	0.789815	+28.24%	10 s
CORAL (this run)	**0.985405**	**+1.50%**	**10 s**
Reference: PyVRP hybrid genetic search	1.000000	0.00%	3 × 60 s

Score = mean over the 50 hidden instances of reference distance / solver distance. Gap is the mean per-instance excess over the reference. The grader passes the same 10-second budget to every program.

Reading of the numbers. CORAL improved the seed score by 0.196 (a 24.8% relative gain), closing the gap to the reference from +28.24% to +1.50%. The winning program is a construction heuristic followed by local search with incremental delta evaluation, discovered by the agents themselves; the task briefing names no algorithm. The reference is not a proven optimum, only a strong one, and it was given roughly eighteen times more compute per instance than the agents were; the remaining 1.5% is the distance to a well-tuned classical solver under a much tighter budget, not the distance to optimality.

Task 3: Ablation study

I ran the experiment 3 times for each of these conditions: **control** (full CORAL), **A** with the notes and skills directories disabled, and **B** with heartbeats disabled. CORAL's `agents.model` is only a runtime alias, so each run's model is verified from the agent logs rather than the config, and any run that drifts off-model or ends early is excluded from the pool rather than silently averaged in.

Implementing the two conditions

Condition A: The `SharingConfig` block reads like the intended switch but is dead code. Nothing in the codebase consults `config.sharing`, so the flag had to be built. A new `agents.knowledge` field was added to `AgentConfig`, together with the plumbing behind it: when false, the four shared-state subdirectories (`notes`, `skills`, `roles`, `agents`) are neither created, seeded, nor symlinked into an agent's worktree; the knowledge passages of `CORAL.md` are factored into separate template fragments and omitted from the generated prompt; and the note-writing heartbeat actions (`consolidate`, `lint_wiki`) are dropped, with `reflect` and `pivot` falling back to note-free variants.

Condition B: `agents.heartbeat=[]` is made to mean what it says, in two files. In `coral/config.py`, `_preprocess` back-fills the default actions only into a partial list, and leaves an explicitly empty one empty. In `coral/hub/heartbeat.py`, `write_agent_heartbeat` and `write_global_heartbeat` take a new `protect` flag, set false when the list is empty, so the protected actions (`reflect`, `consolidate`) are not re-added. No heartbeat is then registered, and none can fire.

Results

Condition	n	Final Score (mean ± sd)	Mean gap
Seed: greedy nearest neighbour	—	0.789815	+28.24%
Control: full CORAL	**3**	**0.9877 ± 0.0024**	**+1.26%**
A: notes/skills disabled	3	0.9805 ± 0.0120	+2.01%
B: heartbeats disabled	3	0.9829 ± 0.0068	+1.77%

Final Score = mean over the 50 hidden instances of reference distance / solver distance, as in Task 2. Three runs per condition, each to the same budget of 12 evaluations.

Reading of the numbers. The three arms are statistically indistinguishable: the largest separation from the control (0.0072, condition A) lies within the between-run dispersion of the conditions themselves (sd 0.0024–0.0120). The likely explanation is that this task leaves the agents little to accumulate. The seed scores 0.789815, yet the first evaluated attempt of almost every run already scores between 0.94 and 0.99, so the greater part of the total improvement is realised before any note, skill or heartbeat can have taken effect. CVRP is a canonical operations-research problem with an extensive published literature, and an agent with web access can therefore retrieve an established construction heuristic (Clarke-Wright savings, a Prins-style optimal split of a giant tour) together with a standard local-

search refinement, and reconstruct a competitive solver at the first attempt. That's why I think even if we run more than 3 times for each condition, the result is still likely to be statistically insignificant.

Task 4: Improve CORAL — knowledge distillation and memory management

1. The failure mode, with evidence

The assessment names three expected problems: noise accumulation, retrieval degradation and redundancy. One of the causes might be that a note becomes visible to every agent the moment it is written, because `notes/` and `skills/` in each worktree are symlinks into the shared state, and nothing evaluates it afterwards.

A note's `status` and `confidence` fields are self-declared, and nothing reads them except a format linter and the dashboard. The only ordering in the retrieval path (`_sort_key`) is the timestamp. The listing an agent sees (`format_notes_list`) prints date, title and creator only. A refuted note and a confirmed one therefore look the same everywhere except inside the file body.

In my experiments, the problems appeared in these forms:

1. **Noise accumulation.** Six notes across the three runs are marked `refuted` by their own authors. Nothing demotes or flags them, and they are served to every later reader in the same way as the confirmed ones.
2. **Redundancy.** In `full-s1` the two agents built the same first improvement separately, Clarke-Wright construction plus local search, and wrote it up as two confirmed notes scoring 0.952658 and 0.952742.

Run	Notes	Refuted	Cite an attempt	Citation resolves	Quoted scores match
full-s1	10	2	9	9/9	9/9
full-s2	6	1	5	5/5	5/5
full-s3	8	3	7	7/7	7/7
Total	**24**	**6**	**21**	**21/21**	**21/21**

Every note in the three full-CORAL runs of Task 3, checked read-only against the same run's attempt store. Note counts exclude the index and housekeeping files. A citation resolves when the cited hash exists in the store as a real, scored attempt whose agent matches the note's creator.

2. The change: a publication gate

The gate runs on its existing monitor loop, over every note that is new or has changed since the last tick. It is aimed at the two problems the evidence shows: notes that cannot be trusted, and notes that repeat each other. Everything in it is plain code: JSON reads, frontmatter parsing, number comparison, and set overlap on tags; there is no LLM call and no embedding model in the path. A model-based judge would be slower, non-deterministic, and itself a new source of wrong verdicts, while every check here is exact and reproducible.

The gate mechanics aim to prevent 2 things:

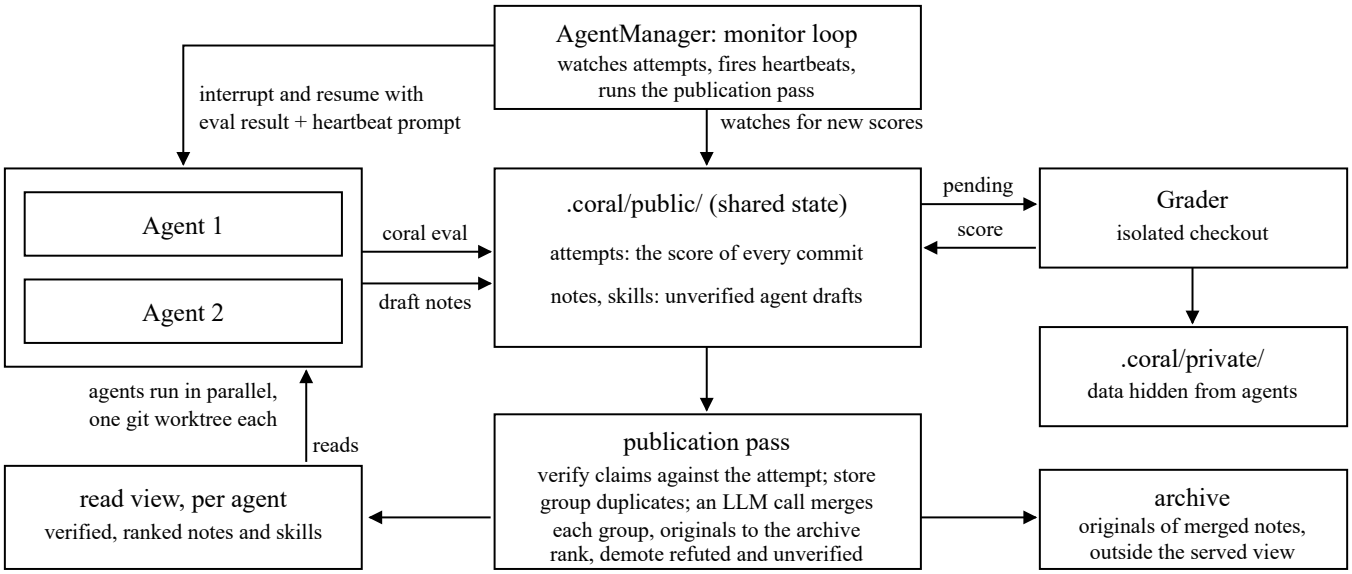
1. **Noise accumulation.** For each note, the pass parses the evidence block and looks the cited hash up in the attempt store. The note is verified only if the hash exists, the attempt is real and scored, the note's creator is the agent that made that attempt, and every score the note quotes equals the recorded one (run offline over the Task 3 corpora, this check confirms the 21 citing notes and leaves the three evidence-free synthesis notes unverified; the 21/21 columns in the table are its output). The verdict is written into the note as computed fields authored by the framework, next to the agent's self-declared ones, and the served list is ordered by computed confidence first, recency second. Notes that fail the check, and notes marked `refuted`, are moved to the bottom of the view with an annotation line; nothing is deleted, so the record of what failed stays readable.
2. **Redundancy.** After verification, the pass looks for duplicates with two rules, both mechanical. Notes that cite the same attempt hash describe the same experiment and are folded into one entry; notes whose frontmatter tag sets overlap beyond a set threshold are grouped, with the best-verified note as the entry and the rest as "see also" lines under it. The duplicated Clarke-Wright pair in `full-s1` is caught by the tag rule, since the two notes cite different attempts but share nearly the same tags. On top of the grouping there is one model-assisted step, added for the experiment below: when a duplicate group forms, a single LLM call drafts one merged note, and the draft is accepted only if it passes the same mechanical verification as any other note — every attempt it cites must resolve in the store with the exact recorded score. On success the originals move to an archive outside the served view, so the agents see only the merged note; on any failure nothing is touched and the group is cached so the call is not repeated. The model only writes prose; what counts as true is still decided by the mechanical checks. A paraphrase written with fresh tags escapes both rules; an embedding-similarity check could be added later, but the duplicates observed in the runs do not need one.

What lands in each worktree is a generated view: `notes/` and `skills/` stop being symlinks and become a directory the manager writes, one file at a time with atomic renames, because under Docker the worktree is a bind mount and a directory swap would not propagate. The index is generated as well, so it cannot link to missing notes or omit real ones, which removes the rot seen in `full-s1`.

Everything else is untouched: attempts, grading, heartbeats and `agents.knowledge=false` (Task 3's condition A) behave as before, and the vanilla arm of the comparison is the unmodified code path.

3. The system after the change

The run loop is unchanged except for the knowledge path. Agents still submit attempts, the grader still scores them against hidden data, and heartbeats still fire. The shared `notes/` directory becomes a pool of unverified drafts, the publication pass joins each draft with the attempt store, and each agent's worktree receives a verified, ranked view instead of a raw symlink.



The run after the change; compare the first figure. Writing is unchanged: agents still put notes and skills into the shared state at will. Reading is not: `notes/` and `skills/` in a worktree are a view the manager generates, and the publication pass checks every claim against the attempt store on the way through; when a duplicate group forms, one LLM call writes the merged note and the originals leave the served view for the archive.

4. Comparative experiment

As I said in Task 3, since this problem is a classic one, most of the meaningful heuristics have already been done pretty well in the first evaluation. Therefore the runs show very little improvement in terms of final score.

Run	Vanilla	Gate + consolidation
s1	0.9854	0.9860
s2	0.9902	0.9933
s3	0.9875	0.9841
mean ± std	**0.9877 ± 0.0024**	**0.9878 ± 0.0048**

Final Score per run and per arm. The vanilla arm reuses the three Task 3 control runs: with the gate flag off, the code path is identical to the one they ran. Runs are independent seeds, not pairs; the row alignment is positional only.

That's why we look at the secondary metrics:

1. **Verification.** The three runs ended with 22 notes in the shared store. 16 were verified against the attempt store and served as such; 6 whose citations could not be checked (no evidence cited, or a cited attempt absent from the store or unscored) were marked unverified and ranked below the verified ones. The generated index stayed correct in all three runs; the hand-kept index rot seen in `full-s1` did not recur.
2. **Consolidation.** Five duplicate groups formed and two were merged by the LLM step, in both cases a pair of notes describing the same line of work (an ILS pair in s2, a perturbation pair in s3). Each merged note passed verification before it replaced anything, the four originals went to the archive, and both agents' views then showed only the merged note.

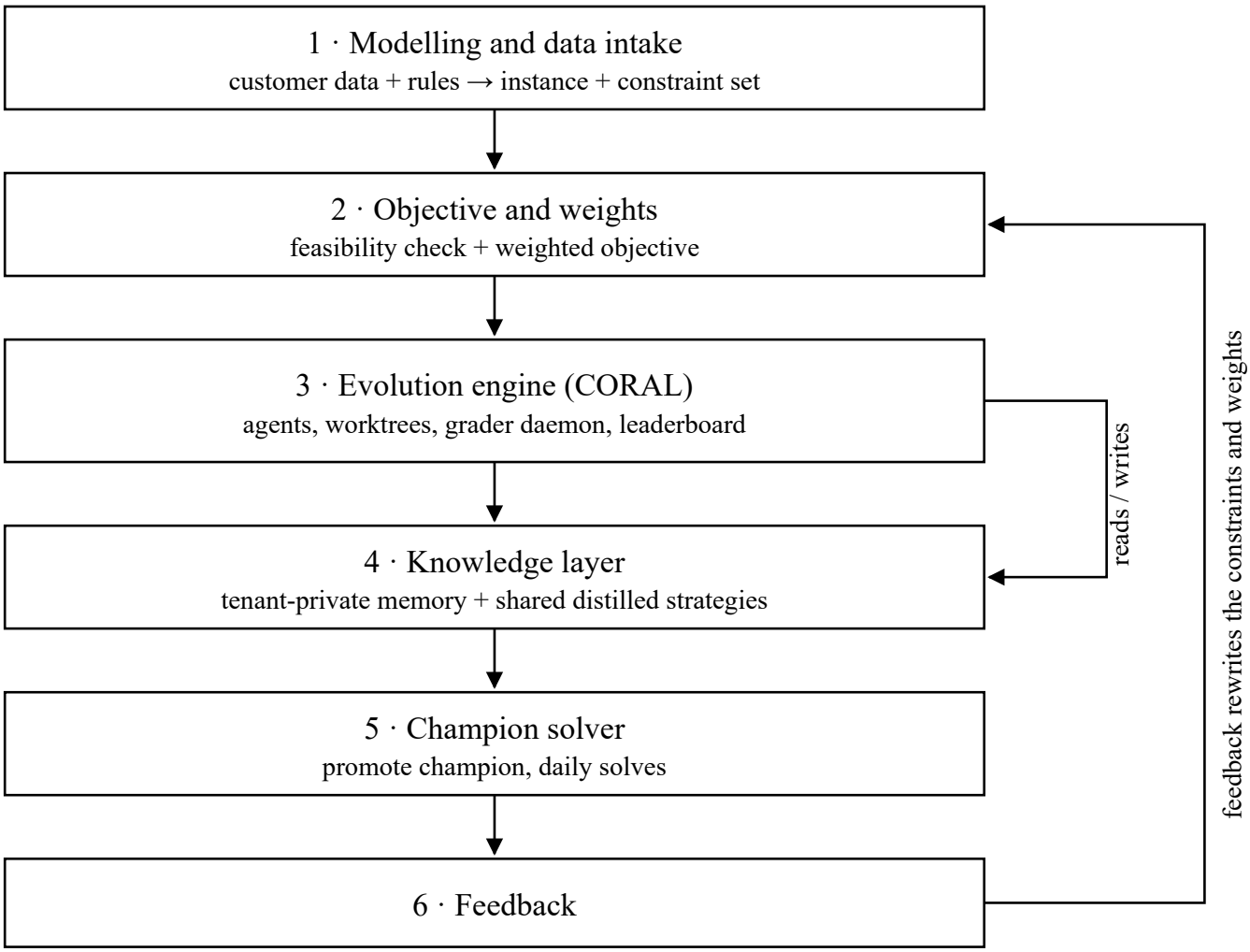
Overall, the changes are score-neutral on this task, as Task 3 predicted, and their effect is on the corpus the agents read: verified notes ranked above unverified ones, a generated index that cannot rot, and duplicate lines of work merged into a single note instead of accumulating.

Task 5: Product plan

1. System architecture

1. **Modelling and data intake:** Customer data and stated rules become a canonical instance and an explicit constraint set. Hard constraints (capacity, licences, legal rest periods) are separated from soft ones (fairness, overtime, preference) here, because that split is what later makes vague feedback actionable.
2. **Objective and weights:** Checks feasibility against the hard constraints and computes a weighted objective over the soft ones. This is the ground truth of the whole system and the only thing the agents optimise against, so it is versioned and owned like production code, not like a config.

3. **Evolution engine (CORAL):** Runs against the tenant's historical instances as the hidden set, scored by the objective from module 2.
4. **Knowledge layer:** Replaces the shared folder: a tenant-private store of what worked on this customer's instances, and a shared, distilled library of solver strategies that carries across tenants without carrying their data. Detailed in sections 2 and 3.
5. **Champion solver:** Find the winning eval/run from the evolution engine, and serve it as the current champion. Run daily solves with the current champion.
6. **Feedback:** Receive feedback from users (both implicit and explicit). The feedback is used to update the weights of the objective, or to add new constraints.

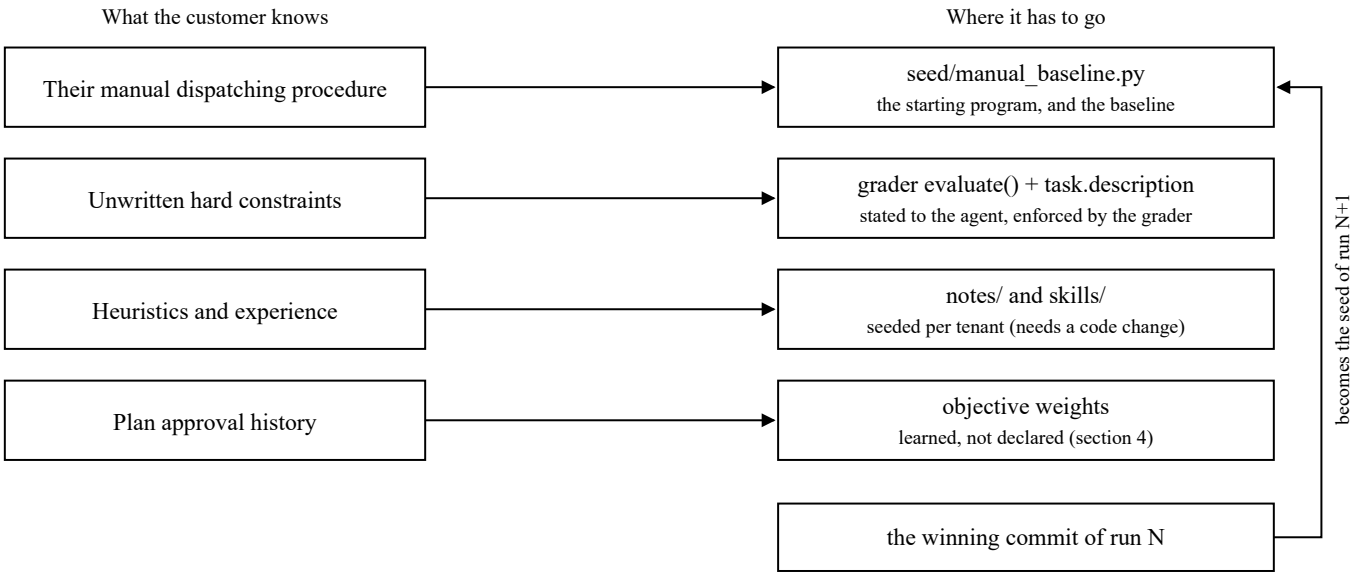


Product module breakdown. All six modules are the product; module 3 is CORAL as it exists today. The loop that makes it a product is the arrow on the right, from user feedback back into the objective and its weights.

2. Knowledge accumulation

CORAL's notes are advisory. No code path reads a note, injects one into an agent's prompt, or verifies that an agent complied with it; agents are told only where the directory is. Knowledge that must be

obeyed cannot be stored there. Customer knowledge is therefore classified by enforcement rather than by importance: knowledge that determines whether a solution is acceptable is compiled into the grader, and knowledge that only accelerates the search is placed in the knowledge channel. This yields four categories with four distinct injection points.



1. **Manual procedure: the seed program.** The dispatcher's current process is implemented as `seed/manual_baseline.py`. Every agent worktree branches from that HEAD, which makes it the common starting point rather than a suggestion.
2. **Unwritten hard constraints: grader and task description.** Any strict rule is implemented as a feasibility check in `TaskGrader.evaluate()` and simultaneously stated in `task.description`, which CORAL renders into the agent's instruction file. If stated but not enforced, the rule is abandoned under score pressure; if enforced but not stated, it must be rediscovered by trial.
3. **Heuristics and domain experience: notes and skills, seeded per tenant.** This is the only category that requires a change to CORAL. `create_project()` constructs an empty `.coral/` on every run and provides no mechanism for carrying knowledge between runs; it must be extended to seed a tenant knowledge pack. Within the pack, executable skills rank above prose notes: a skill, such as a script that queries the customer's travel-time matrix, is invoked and measured, whereas a note may never be read.
4. **Plan approval history: objective weights.** The record of which plans were accepted, edited or rejected is a signal for weight estimation and is done in module 2.

Accumulation between runs is carried by the program, not by the notes. On completion of a run, `coral export` promotes the winning commit to the seed of the next run, so the artefact retained is

the one that was measured. A note is carried into the next run's pack only if its claim cites an attempt hash that exists and whose recorded score matches, a check CORAL does not currently perform.

3. Multi-tenant isolation

I have not deployed a system of this kind for enterprise customers, so I will not pretend that I know the detailed tech stack for this part. But I will try to describe what I think should be private and what should be shared across tenants.

A piece of knowledge is general if it applies to any instance of the problem, and private if it depends on one customer's instance. So the knowledge that should be shared is the one that can help optimise the solver searching technique and searching time like: solver operators, savings insertion and 2-opt; search control schedules; evaluation-speed engineering such as incremental scoring and caching; and defects in the platform itself.

Not shared are the instance data, the grader and the objective weights, which together encode the customer's operating rules, the champion solver, which was selected on that customer's instances and may carry their constants, and the solving history and notes. Notes cannot cross at all, because a note is free text written by an agent. The boundary between the two categories is settled by a test rather than by judgement. A candidate leaves a tenant only as executable code containing no literal drawn from that tenant's instances, and it enters the shared library only if it reproduces its gain on public benchmark instances that the tenant never saw. Knowledge that still improves results on unfamiliar instances was not about the customer; knowledge that fails there was a description of the customer, and it is precisely what must not cross.

4. Fuzzy user feedback

Statements such as "the drivers are too tired" are not defect reports against one schedule; they are evidence that the objective is mis-weighted. Feedback is therefore first routed, not answered. If the plan could not have been executed at all, the model is missing a constraint and the correction belongs in modules 1 and 2. If the plan was executable but unwanted, the objective priced something wrongly and the correction is a weight. Only the second class drives evolution.

The path is not a set of independent channels but a single pipeline of three stages: a divergence is detected, a question is asked about that specific divergence, and the answer is classified.

1. **Detection.** Dispatchers edit a plan before releasing it. The planning interface records both the plan that was served and the plan that was actually released. Each divergence between them is, in itself, an inequality over the objective's weights: under the true objective, the plan the dispatcher chose scores at least as well as the plan that was proposed. The inequality is obtained at no cost in user

effort and is undistorted by phrasing, but it is silent about intent. Its second and more important function is to localise the omission: it identifies the one decision at which an unstated rule took effect.

2. **Interrogation.** The knowledge governing routine dispatching is enacted daily, and familiarity renders it self-evident to the expert and therefore not worth stating, so open requests of the form "is there anything else we should know" under-report precisely the rules that carry the most weight. The omission is not evasion but an ordinary consequence of expertise, known in expert-systems practice as the knowledge-acquisition bottleneck. Knowledge is obtained only when the question is specific enough to touch the rule in question; the divergence recorded in stage 1 supplies that specificity, and is put back to the dispatcher as a contrastive question about that case, for example "stop 14 was moved from route B to route C before release; what made route B unusable?". The record localises the unstated rule; the question extracts it.
3. **Classification.** The answer returns as natural language, as does any unprompted comment left against a plan. A model maps that language onto the objective's existing terms: which term is implicated (driver hours, overtime, distance, lateness, fairness), in which direction, and at what scope (this instance, this depot, always). "The drivers are too tired" becomes a proposed increase in the penalty on driver hours. The model performs classification only. It never sets a number and never writes to the grader; it emits a proposal, which enters the weight estimation alongside every other observation.

A caution on incumbency. Every instrument above learns from what the dispatchers do, and the dispatchers have done the same thing for years. Left unguarded the loop amplifies incumbency: an unfamiliar plan is rejected because it is unfamiliar, the rejection is read as a revealed preference, the objective is adjusted to penalise whatever made the plan unfamiliar, and the system converges on the practice it was procured to improve. The loss would not appear as a loss, because the objective that would have measured it has itself moved.

5. Foreseeable failure modes

I think most deployments of this kind fail because the problem was modelled wrongly, not because the search was weak.

Failure 1. Model and data

Problem

The objective may not state what the customer actually wants. The grader returns a high score, the dispatcher calls the plan unusable and fixes it by hand, and the measured gain is lost, because that plan breaks a rule nobody wrote down. The data may be wrong and the solver will optimise them

faithfully, with wrong coordinates. For example it can be an out of date travel time matrix, or a vehicle capacity that the fleet does not actually meet.

Strategy to mitigate

We need constant discussion with the customer to ensure the objective states what the customer actually wants. As for data accuracy, we want a checking layer that runs before the solver and rejects bad input rather than repairing it. Every rule the customer states is tested against the record of past approved plans before it is accepted, because a rule that a fair number of executed plans already break is a preference and should enter as a weight. If we write it into the grader as a hard rule instead, it would cut away part of the legal solution space, and the agent could never again find the better plan that was inside it.

Failure 2. The self-directed agent

Problem

The agent can read exactly how it is scored. In CORAL the grading code sits in the task's `grader/` folder and is shown to every agent as read-only, and this stays true even when the agent runs inside a sandbox or a container, because only the hidden data, not the scoring code, is placed behind that wall. So an agent that writes its own program, sees its own score, and can read the code that produced it will, after several hundred rounds, drift toward the cheapest way to raise the number. Often the cheapest way is not to solve the problem but to cheat the measurement. It may store answers for instances it has already seen, read a known optimum from a value in the input, use a rounding or tie-breaking detail in the grader, or special-case the few visible development instances. The score is then high and the solver is useless.

Strategy to mitigate

Because the scoring code cannot be hidden, we assume the agent knows it in full and build the harness so that the only way to raise the score is to actually solve new instances. First, we keep a holdout set the agent never sees, and the gap between the development score and the holdout score tells us how far the agent has fitted the grader instead of the problem. Second, the promotion gate scores the champion again on instances created after it was written, and probes change an instance in ways that must not change the answer, so a program that only memorised or special-cased earlier cases is exposed. This does not eliminate the risk entirely, but it makes it harder for the agent to cheat the measurement.

Task 6: Reflections

This week of experiments and iteration has been a genuinely valuable exercise. In my opinion, Task 2 should have been built on a less classical problem, so that the evolutionary behaviour of the agents

would have had more room to show itself. Overall, the tasks taught me a good deal: how to orchestrate the agents, and how objectives and knowledge can be brought into the agents' consideration by adjusting the weights and the objective.

I also want to say that I am a recent graduate in Data Science, and my expertise does not yet match that of those with years of industry experience. However I can learn things quickly, and I am confident that I can catch up with whatever the work requires.